

CS-302 Artificial Intelligence

Experiment # 2

Understanding and implementing key concepts of AI

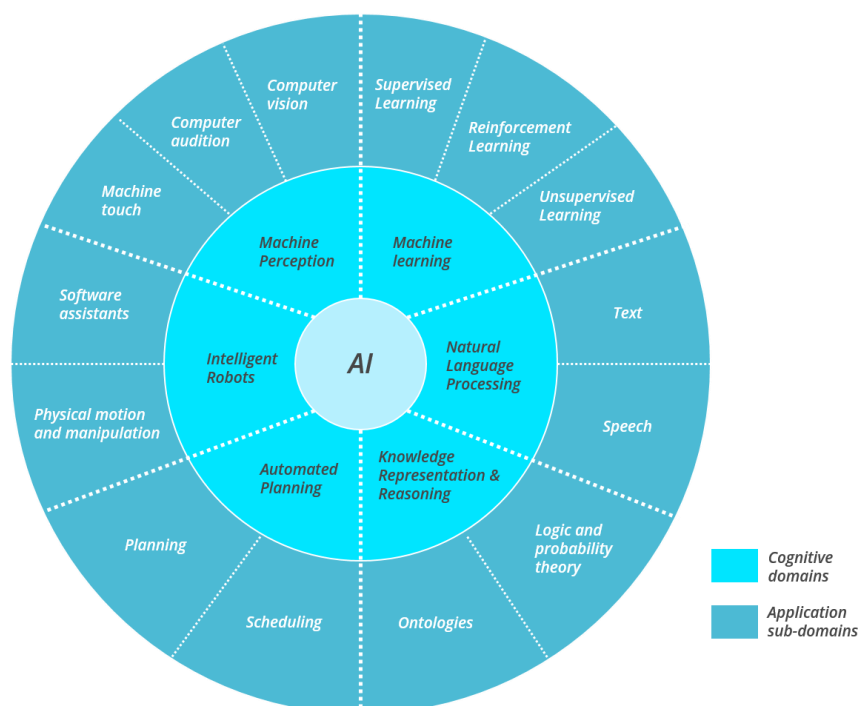
Objective/s:

- Understanding AI
- Knowing AI domains and corresponding sub domains
- Machine learning
- Solving some elementary example tasks
- Lab Tasks

1. Understanding AI

“Artificial Intelligence (AI) is the ability of a computer-controlled entity to perform cognitive tasks and react flexibly to its environment in order to maximize the probability of achieving a particular goal. The system can learn from experience data, and can mimic behaviors associated to humans, but does therefore not necessarily use methods that are biologically observable.”

2. AI classification



- Machine learning
- Knowledge representation and reasoning
- Automated Planning
- Natural language processing
- Machine Perception
- Intelligent Robots

The second layer shows (almost certainly not exhaustive) corresponding sub-domains, in which the major six categories can be functionally categorized. But it is getting messy already, because some disciplines interact very closely like for example perception and natural language processing in many areas of applications (translation or your Amazon Echo speaker).

For better understanding we will stick to Machine Learning domain for now and further explore its corresponding problems. We will also try to solve some elementary problems for hands on experience.

3. Machine learning

Machine learning is behind chatbots and predictive text, language translation apps, the shows Netflix suggests to you, and how your social media feeds are presented. It powers autonomous vehicles and machines that can diagnose medical conditions based on images.

When companies today deploy artificial intelligence programs, they are most likely using machine learning — so much so that the terms are often used interchangeably, and sometimes ambiguously. Machine learning is a subfield of artificial intelligence that gives computers the ability to learn without explicitly being programmed.

3.1. What is machine learning

Machine learning is a subfield of artificial intelligence, which is broadly defined as the capability of a machine to imitate intelligent human behavior. Artificial intelligence systems are used to perform complex tasks in a way that is similar to how humans solve problems.

The goal of AI is to create computer models that exhibit “intelligent behaviors” like humans, according to Boris Katz, a principal research scientist and head of the InfoLab Group at CSAIL. This means machines that can recognize a visual scene, understand a text written in natural language, or perform an action in the physical world.

3.2. How machine learning works

Machine learning starts with data — numbers, photos, or text, like bank transactions, pictures of people or even bakery items, repair records, time series data from sensors, or sales reports. The data is gathered and prepared to be used as training data, or the information the machine learning model will be trained on. The more data, the better the program.

From there, programmers choose a machine learning model to use, supply the data, and let the computer model train itself to find patterns or make predictions. Over time the human programmer can also tweak the model, including changing its parameters, to help push it toward more accurate results

3.3. Subcategories of machine learning

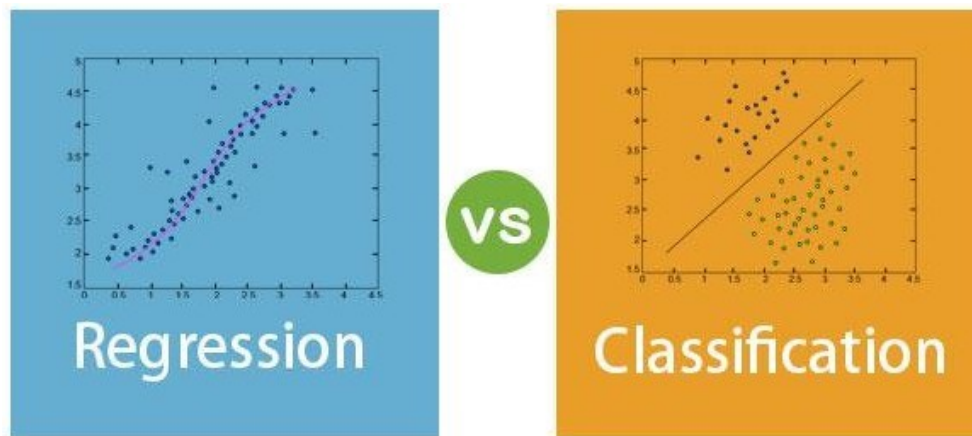
There are three subcategories of machine learning

3.3.1. Supervised learning

Supervised machine learning models are trained with labeled data sets, which allow the models to learn and grow more accurate over time. For example, an algorithm would be trained with pictures of dogs and other things, all labeled by humans, and the machine would learn ways to identify pictures of dogs on its own. Supervised machine learning is the most common type used today.

Regression and Classification algorithms are Supervised Learning algorithms. Both the algorithms are used for prediction in Machine learning and work with the labeled datasets. But the difference between both is how they are used for different machine learning problems.

The main difference between Regression and Classification algorithms that Regression algorithms are used to predict the continuous values such as price, salary, age, etc. and Classification algorithms are used to predict/Classify the discrete values such as Male or Female, True or False, Spam or Not Spam, etc.



3.3.1.1. Classification:

Classification is a process of finding a function which helps in dividing the dataset into classes based on different parameters. In Classification, a computer program is trained on the training dataset and based on that training, it categorizes the data into different classes.

The task of the classification algorithm is to find the mapping function to map the input(x) to the discrete output(y).

Example: The best example to understand the Classification problem is Email Spam Detection. The model is trained on the basis of millions of emails on different parameters, and whenever it receives a new email, it identifies whether the email is spam or not. If the email is spam, then it is moved to the Spam folder.

Classification Algorithms can be further divided into the following types:

- Logistic Regression
- K-Nearest Neighbours
- Support Vector Machines
- Kernel SVM
- Naïve Bayes
- Decision Tree Classification
- Random Forest Classification

3.3.1.2. Regression:

Regression is a process of finding the correlations between dependent and independent variables. It helps in predicting the continuous variables such as prediction of Market Trends, prediction of House prices, etc.

The task of the Regression algorithm is to find the mapping function to map the input variable(x) to the continuous output variable(y).

Example: Suppose we want to do weather forecasting, so for this, we will use the Regression algorithm. In weather prediction, the model is trained on the past data, and once the training is completed, it can easily predict the weather for future days.

- Simple Linear Regression
- Multiple Linear Regression
- Polynomial Regression
- Support Vector Regression
- Decision Tree Regression
- Random Forest Regression

3.3.2. Unsupervised learning

In unsupervised machine learning, a program looks for patterns in unlabeled data. Unsupervised machine learning can find patterns or trends that people aren't explicitly looking for. For example, an unsupervised machine learning program could look through online sales data and identify different types of clients making purchases.

3.3.3. Reinforcement learning

Reinforcement machine learning trains machines through trial and error to take the best action by establishing a reward system. Reinforcement learning can train models to play games or train autonomous vehicles to drive by telling the machine when it made the right decisions, which helps it learn over time what actions it should take.

4. Example task

The best small project to start with on a new tool is the classification of iris flowers (e.g. the iris dataset).

This is a good project because it is so well understood.

- Attributes are numeric so you have to figure out how to load and handle data.
- It is a classification problem, allowing you to practice with perhaps an easier type of supervised learning algorithm.
- It is a multi-class classification problem (multi-nominal) that may require some specialized handling.
- It only has 4 attributes and 150 rows, meaning it is small and easily fits into memory (and a screen or A4 page).
- All of the numeric attributes are in the same units and the same scale, not requiring any special scaling or transforms to get started.

Let's start solving our first classification or in fact first machine learning task in next section.

5. Complete guide on building a new project in python.

1. Install SciPy Libraries

There are 5 key libraries that you will need to install. Below is a list of the Python SciPy libraries required for this tutorial:

- scipy
- numpy
- matplotlib
- pandas
- sklearn

2. Load the dataset

We are going to use the iris flowers dataset. This dataset is famous because it is used as the “hello world” dataset in machine learning and statistics by pretty much everyone.

The dataset contains 150 observations of iris flowers. There are four columns of measurements of the flowers in centimeters. The fifth column is the species of the flower observed. All observed flowers belong to one of three species.

We can load the data directly from the UCI Machine Learning repository.

We are using pandas to load the data. We will also use pandas next to explore the data both with descriptive statistics and data visualization.

```
...  
# Load dataset  
url =  
"https://raw.githubusercontent.com/jbrownlee/Datasets/master/iris.csv"  
names = ['sepal-length', 'sepal-width', 'petal-length', 'petal-width',  
         'class']  
dataset = read_csv(url, names=names)
```

Note that we are specifying the names of each column when loading the data. This will help later when we explore the data.

3. Evaluate some models

Now it is time to create some models of the data and estimate their accuracy on unseen data.

Here is what we are going to cover in this step:

- Separate out a validation dataset.
- Set-up the test harness to use 10-fold cross validation.
- Build multiple different models to predict species from flower measurements
- Select the best model.

3.1. Create a Validation Dataset

We need to know that the model we created is good.

Later, we will use statistical methods to estimate the accuracy of the models that we create on unseen data. We also want a more concrete estimate of the accuracy of the best model on unseen data by evaluating it on actual unseen data.

That is, we are going to hold back some data that the algorithms will not get to see and we will use this data to get a second and independent idea of how accurate the best model might actually be.

We will split the loaded dataset into two, 80% of which we will use to train, evaluate and select among our models, and 20% that we will hold back as a validation dataset.

```
# Split-out validation dataset
array = dataset.values
X = array[:,0:4]
y = array[:,4]
X_train, X_validation, Y_train, Y_validation = train_test_split(X, y,
test_size=0.20, random_state=1)
```

3.2. Build models

We don't know which algorithms would be good on this problem or what configurations to use.

We get an idea from the plots that some of the classes are partially linearly separable in some dimensions, so we are expecting generally good results.

Let's test 6 different algorithms:

- Logistic Regression (LR)
- Linear Discriminant Analysis (LDA)
- K-Nearest Neighbors (KNN).
- Classification and Regression Trees (CART).
- Gaussian Naive Bayes (NB).

- Support Vector Machines (SVM).

Lets build and evaluate models

```
...
# Spot Check Algorithms
models = []
models.append(('LR', LogisticRegression(solver='liblinear',
multi_class='ovr'))))
models.append(('LDA', LinearDiscriminantAnalysis()))
models.append(('KNN', KNeighborsClassifier()))
models.append(('CART', DecisionTreeClassifier()))
models.append(('NB', GaussianNB()))
models.append(('SVM', SVC(gamma='auto'))))
# evaluate each model in turn
results = []
names = []
for name, model in models:
    kfold = StratifiedKFold(n_splits=10, random_state=1, shuffle=True)
    cv_results = cross_val_score(model, X_train, Y_train, cv=kfold,
scoring='accuracy')
    results.append(cv_results)
    names.append(name)
    print('%s: %f (%f)' % (name, cv_results.mean(), cv_results.std()))
```

3.3. Select best model

We now have 6 models and accuracy estimations for each. We need to compare the models to each other and select the most accurate.

Running the example above, we get the following raw results:

```
LR: 0.960897 (0.052113)
LDA: 0.973974 (0.040110)
KNN: 0.957191 (0.043263)
CART: 0.957191 (0.043263)
NB: 0.948858 (0.056322)
SVM: 0.983974 (0.032083)
```

4. Make Predictions

We must choose an algorithm to use to make predictions.

The results in the previous section suggest that the SVM was perhaps the most accurate model. We will use this model as our final model.

Now we want to get an idea of the accuracy of the model on our validation set.

This will give us an independent final check on the accuracy of the best model. It is valuable to keep a validation set just in case you made a slip during training, such as overfitting to the training set or a data leak. Both of these issues will result in an overly optimistic result.

```
# make predictions
from pandas import read_csv
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score
from sklearn.svm import SVC

# Load dataset
url="https://raw.githubusercontent.com/jbrownlee/Datasets/master/iris.csv"
names = ['sepal-length', 'sepal-width', 'petal-length', 'petal-width',
'class']
dataset = read_csv(url, names=names)

# Split-out validation dataset
array = dataset.values
X = array[:,0:4]
y = array[:,4]
X_train, X_validation, Y_train, Y_validation = train_test_split(X, y,
test_size=0.20, random_state=1)

# Make predictions on validation dataset
model = SVC(gamma='auto')
model.fit(X_train, Y_train)
predictions = model.predict(X_validation)

# Evaluate predictions
print(accuracy_score(Y_validation, predictions))
print(confusion_matrix(Y_validation, predictions))
print(classification_report(Y_validation, predictions))
```


6. Tasks

1. Write the above mentioned program for iris dataset